

Toward Intelligent, Reliable, and Secure Automation of CI/CD Workflows

Jiangnan Huang

Institute for Computing and Information Sciences (iCIS)

Radboud University

Nijmegen, Netherlands

jiangnan.huang@ru.nl

Abstract

CI/CD pipelines form a core component of contemporary software engineering, automating key stages of the development lifecycle. Since its release in 2019, GitHub Actions (GHA) has rapidly emerged as the dominant CI/CD platform on GitHub, automating millions of workflows daily. However, as the ecosystem grows, developers face persistent challenges in automating GHA workflow creation, mitigating workflow failures, and ensuring security. To address these challenges, my doctoral research systematically examines how developers construct, maintain, and secure GHA workflows in practice, and proposes learning-based approaches to improve their automation quality. The research includes two empirical studies on workflow reliability (reruns and flakiness) and security (evolution of developers' practices), along with two learning-based frameworks: CIGAR for recommending reusable Actions and FLOWSTEP for generating step-aware workflows. Together, they aim to build an *intelligent and secure automation framework* for GitHub Actions integrating reliability, security, and semantic understanding.

CCS Concepts

• **Software and its engineering** → **Software development methods**.

Keywords

GitHub Actions, CI/CD, Automation, Security, Reliability

ACM Reference Format:

Jiangnan Huang. 2026. Toward Intelligent, Reliable, and Secure Automation of CI/CD Workflows. In *2026 IEEE/ACM 48th International Conference on Software Engineering (ICSE-Companion '26)*, April 12–18, 2026, Rio de Janeiro, Brazil. ACM, New York, NY, USA, 3 pages. <https://doi.org/10.1145/3774748.3787630>

1 Introduction

Continuous Integration and Continuous Delivery (CI/CD) is vital for automating software testing, building, and deployment [15]. Since its release in 2019, GitHub Actions¹ (GHA) has become the dominant CI/CD solution on GitHub, favored for its seamless integration, flexible workflow design, and vast library of community-contributed *Actions* [6], making it the most representative platform

¹<https://github.com/features/actions>



This work is licensed under a Creative Commons Attribution 4.0 International License. *ICSE-Companion '26*, Rio de Janeiro, Brazil

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2296-7/26/04

<https://doi.org/10.1145/3774748.3787630>

for CI/CD workflows studies. Despite its popularity, GHA faces key challenges: (i) limited *tool support*, as workflow constructing and maintenance remain largely manual, requiring developers to compose YAML pipelines and configure actions by hand [8, 12, 16]; (ii) *reliability* issues such as flaky runs and redundant reruns that waste resources; and (iii) *security* risks from unverified *Actions*, secret leaks, and misconfigured permissions [3, 9, 13]. To bridge these gaps, my dissertation focuses on the following research question:

How can we automate the construction of GHA workflows and improve their reliability and security?

This research began with CIGAR [8], a contrastive-learning model recommending relevant *Actions* from user-assigned task names, showing the promise of learning-based workflow assistance. After which, two large-scale studies are conducted on workflow reliability (reruns) and security evolution [9]. Building on these insights, we are developing FLOWSTEP, an LLM-driven framework for intelligent and secure workflow generation.

2 Background and Related Work

Automation and Learning-Based Assistance. RSSEs (recommendation systems for software engineering) are widely used across development tasks, but mainly target source code generation [1]. With GHA's growth, learning-based methods have emerged for workflow generation using models such as T5 and LLMs [12, 16], though often overlooking semantic differences between step types. Meanwhile, LLM-based evaluation offers new potential for assessing functional correctness of the generated workflows [4].

GitHub Actions and Workflow Reliability. A GHA workflow of YAML format comprises *jobs* triggered by *events*, each containing *steps* that invoke *Actions* (i.e., reusable components that perform defined tasks) or execute shell *Scripts*. Like other CI/CD systems (e.g., Travis CI, Jenkins), GHA workflows may exhibit non-determinism due to flaky tests, infrastructure variance, or dependency instability. While test flakiness has been widely studied [14], workflow-level reruns and flakiness remains largely underexplored.

Workflow Security and Developer Practices. Recent studies highlight major GHA vulnerability classes [11], identify misconfigurations via static analysis [3], and detect command injections through taint-based analysis [13]. Others report flaws in JavaScript actions [7], recurring workflow “smells” [10], and low adoption of best practices [2]. However, the majority of existing security studies remain cross-sectional, offering one-time snapshots of the ecosystem rather than systematically examining how workflow security evolves over time.

3 Research Progress

My dissertation comprises four main threads that address the automation, reliability, and security of GHA workflows.

3.1 CIGAR: GHA Action Recommendation

3.1.1 Approach. Early in my research, we observed that GitHub Marketplace’s search mechanism often produces imprecise or incomplete results, making it challenging for developers to identify the *Actions* they need. CIGAR [8] (Contrastive learning for GitHub Action Recommendation) is a RoBERTa-based model that suggests relevant *Actions* from developers’ task descriptions. It encodes step names and official *Action* metadata (*i.e.*, official name, description) into vector representations and fine-tunes the encoder with contrastive learning to align semantically related pairs. The dataset, derived from secos-gha (Decan *et al.*, 2022) [6], includes 25k step names and 756 *Actions* from 24k repositories, split for training, validation, and testing. Performance was evaluated via success rate@N and catalog coverage, compared with RoBERTa baselines and GitHub Marketplace’s search mechanism.

3.1.2 Findings. CIGAR achieved strong accuracy (72.4% success@1, 95.5% success@5) and nearly full catalog coverage ($\geq 95\%$), substantially outperforming GitHub Marketplace search (1.8% success@1). Ablation study shows that using both name and description features yielded better results than either alone, and fine-tuning plus contrastive learning substantially improved performance. Overall, the study highlights the effectiveness of contrastive learning for *Action* recommendation and its potential to support CI/CD workflow composition. This work was published in IEEE SCAM’23 [8], where it received the Best Paper Award.

3.2 GHA Workflow Reliability

3.2.1 Approach. During CIGAR’s data collection, frequent reruns of failed workflows were observed. To understand the nature of these reruns and support developers in addressing them, we analyzed 9,202 workflows from 3,320 Java repositories (≥ 100 stars, ≥ 100 commits) using SEART [5] and the GitHub REST API, examining rerun frequency, flakiness causes, and predictive modeling. Three research questions guided the analysis: (i) how often reruns occur, (ii) what causes flakiness, and (iii) whether outcomes can be predicted. For (iii), we trained four lightweight classifiers (Naive Bayes, Decision Tree, SVM, and MLP) on 50k rerun records using historical, complexity, and churn features, benchmarked against All-Success and Straw-Man baselines.

3.2.2 Findings. Reruns are pervasive, 92% of failed workflows are re-executed, with 24.6% full workflow reruns and 76.4% job-level reruns. Job reruns are more effective (73.6% vs. 33.9% success) and faster (0.9 h vs. 3.9 h). In 2023 alone, reruns consumed roughly 3.7 years of waiting and 4.5 years of computation time. Major causes include test flakiness (32–42%), dependency failures, and misconfigurations. Lightweight models such as Decision Tree achieved up to 0.90 F1-score, with historical data as the most predictive feature. The detailed findings of this work have been submitted to TOSEM and are currently under revision.

3.3 GHA Workflow Security Evolution

3.3.1 Approach. Developers increasingly rely on GitHub Actions for critical automation, which raises the question of how workflow security evolves and whether longstanding vulnerabilities continue to persist. Building on our reliability study, we revisited the security of GHA workflows overtime. Using and extending ARGUS dataset [13], we collected 18,938 workflows from 5,246 repositories present in both 2022 and 2024. We then applied GHASt [3] to detect four issue types, including two vulnerabilities (*Confidential Data Disclosure*, *Command Injection*) and two misconfigurations (*Unverified Action Version*, *Missing Permissions*), while categorizing workflow trigger events by exploitability level. Temporal differences were tested using Wilcoxon’s Signed Ranks Test.

3.3.2 Findings. Across 2024 workflows, 33,362 issues were found (2,731 vulnerabilities, 30,631 misconfigurations). *Confidential Data Disclosure* and *Command Injection* remained unchanged ($p \geq 0.05$), while permission hygiene improved significantly (84.7% $\rightarrow$ 74.9%, $p < 0.05$). Risky trigger events declined slightly, and unverified actions (~16k cases) persisted. Overall, developers improved configuration hygiene but made limited progress on core vulnerabilities. This work was published in ICPC’26 ERA track [9].

4 Proposed Research Plan

4.1 Automated Workflow Step Generation

Motivated by prior empirical studies, this work extends CIGAR into FLOWSTEP, an end-to-end framework that models step-type semantics and generates complete workflow steps.

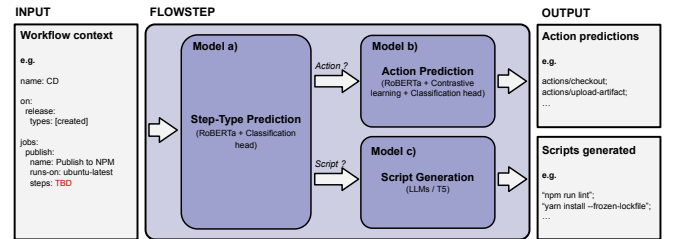


Figure 1: Overall framework of FLOWSTEP

4.1.1 Approach. FLOWSTEP aims to provide modular, learning-based automation for GHA workflow construction. It explicitly distinguishes between *Action* and *Script* steps, reflecting developers’ real-world authoring behavior. The framework will include three core components: (i) a *Step-Type Classifier*, a RoBERTa-based binary model that predicts whether the next step should be an *Action* or *Script*; (ii) an *Action Predictor*, an enhanced version of CIGAR that recommends relevant *Actions* from workflow context; and (iii) a *Script Generator*, which employs LLMs (*e.g.*, Claude-Sonnet-4.5) to produce inline scripts when applicable, as illustrated in Fig. 1.

We adopted the training corpus from Mastropaolo *et al.* [12], which is built upon secos-gha [6], comprising nearly 18k unique workflows. Both raw and abstracted variants are used: in the abstracted form, non-generalizable tokens (*e.g.*, paths, URLs) are replaced with placeholders to enable reproducible evaluation and

fair comparison with prior work. The framework will be evaluated through standard metrics (Top-K accuracy, BLEU, ROUGE) and the proposed *LLM-as-a-Judge* evaluation, where LLMs assess the semantic correctness of generated scripts beyond surface similarity.

4.1.2 Expected Outcomes. FLOWSTEP is expected to advance automated workflow construction by integrating step-type awareness and semantics-based evaluation. Preliminary experiments indicate notable performance gains, suggesting that explicit modeling of step types may improve end-to-end raw accuracy from roughly 39% to 50%. The enhanced CIGAR is expected to outperform prior *Action* predictors by approximately 15%. For *Script* generation, we anticipate LLM-based models to produce more functionally coherent outputs than earlier baselines. Finally, the LLM-as-a-Judge paradigm is expected to provide semantics-aware, human-aligned evaluation complementary to traditional text-based metrics.

4.2 Future Directions

Building upon the FLOWSTEP framework, the next phase of this dissertation will focus on extending workflow generation to support reliability- and security-aware automation. Specifically, we plan to integrate reasoning mechanisms that incorporate insights from earlier empirical studies, including common misconfigurations, known vulnerability patterns, and historical failure signals. This enhancement will allow FLOWSTEP not only to generate functionally correct steps but also to proactively identify risky constructs and recommend safer alternatives. This will ultimately pave the way toward self-healing and trustworthy workflow automation.

In parallel, a systematic literature survey on learning-based CI/CD assistance is currently ongoing to consolidate the state of the art in this domain. The survey reviews existing approaches to CI/CD workflow generation, recommendation, and repair, examining how learning-based and LLM-driven techniques have been applied to CI/CD automation. Its objectives are threefold: to classify research trends, to identify methodological and practical gaps, and to position FLOWSTEP within the broader ecosystem of intelligent DevOps tools and frameworks.

5 Conclusions and Contributions

This dissertation advances software engineering by offering an integrated view of the **automation**, **reliability**, and **security** of GitHub Actions workflows. Its core contributions are:

- **Workflow Automation.** We introduce two learning-based frameworks for workflow automation: CIGAR, which uses contrastive learning for Action recommendation, and FLOWSTEP, which adds step-type awareness and leverages state-of-the-art LLMs for step completion, yielding substantial accuracy and semantic gains.
- **Workflow Reliability.** We conduct the first large-scale study of GHA workflow reruns, uncovering their resource waste and underlying flakiness. Leveraging these insights, we develop lightweight models to predict execution outcomes and reduce unnecessary reruns.
- **Workflow Security.** We enhance the GHA tool to support cross-repository analysis of workflow security evolution, showing that although configuration hygiene improves,

persistent vulnerabilities such as command injection warrant continued attention.

All tools, models, and datasets developed in these works will be made publicly available to support open science and reproducibility. Together, these contributions converge toward a unified understanding of how AI can assist the construction of CI/CD workflows, how such workflows fail in real-world settings, and how to secure them, laying a foundation for the next generation of reliable, secure, and intelligent CI/CD automation.

6 Timeline for Completion

We plan to complete the evaluation of the FLOWSTEP framework in the first quarter of 2026. Subsequent work will focus on secured workflow generation and the planned survey, with the dissertation defense scheduled at the end of 2026 or the beginning of 2027.

References

- [1] 2016. What recommendation systems for software engineering recommend: A systematic literature review. *Journal of Systems and Software* 113 (2016), 101–113.
- [2] Jessy Ayala and Joshua Garcia. 2023. An Empirical Study on Workflows and Security Policies in Popular GitHub Repositories. In *2023 IEEE/ACM 1st International Workshop on Software Vulnerability (SVM)*. 6–9.
- [3] Giacomo Benedetti, Luca Verderame, and Alessio Merlo. 2022. Automatic Security Assessment of GitHub Actions Workflows. ACM, New York, NY, USA.
- [4] Giuseppe Crupi, Rosalia Tufano, Alejandro Velasco, Antonio Mastropaolo, Denys Poshyvanyk, and Gabriele Bavota. 2025. On the Effectiveness of LLM-as-a-judge for Code Generation and Summarization. arXiv:2507.16587 [cs.SE]
- [5] Ozren Dabic, Emad Aghajani, and Gabriele Bavota. 2021. Sampling Projects in GitHub for MSR Studies. In *18th IEEE/ACM International Conference on Mining Software Repositories, MSR 2021*. IEEE, 560–564.
- [6] Alexandre Decan, Tom Mens, Pooya Rostami Mazrae, and Mehdi Golzadeh. 2022. On the Use of GitHub Actions in Software Development Repositories. In *2022 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. 235–245.
- [7] Hassan Onsoni Delicheh, Alexandre Decan, and Tom Mens. 15 April 2024. Quantifying Security Issues in Reusable JavaScript Actions in GitHub Workflows. In *21st International Conference on Mining Software Repositories (MSR '24)* (Lisbon, Portugal). F.R.S.-FNRS - Fonds de la Recherche Scientifique [BE], ACM.
- [8] Jiangnan Huang and Bin Lin. 2023. CIGAR: Contrastive Learning for GitHub Action Recommendation. In *2023 IEEE 23rd International Working Conference on Source Code Analysis and Manipulation (SCAM)*. 61–71.
- [9] Jiangnan Huang and Bin Lin. 2025. Revisiting Security Practices for Github Actions Workflows. In *2025 IEEE/ACM 33rd International Conference on Program Comprehension (ICPC)*. 73–77.
- [10] Ali Khatami, Cédric Willekens, and Andy Zaidman. 2024. Catching smells in the act: A github actions workflow investigation. In *24th IEEE International Working Conference on Source Code Analysis and Manipulation (SCAM)*. IEEE.
- [11] Igibek Koishybayev, Aleksandr Nahapetyan, Raima Zachariah, Siddharth Muralee, Bradley Reaves, Alexandros Kapravelos, and Aravind Machiry. 2022. Characterizing the Security of Github CI Workflows. In *31st USENIX Security Symposium (USENIX Security 22)*. USENIX Association, Boston, MA, 2747–2763.
- [12] Antonio Mastropaolo, Fiorella Zampetti, Gabriele Bavota, and Massimiliano Di Penta. 2024. Toward Automatically Completing GitHub Workflows. ACM, New York, NY, USA.
- [13] Siddharth Muralee, Igibek Koishybayev, Aleksandr Nahapetyan, Greg Tystahl, Brad Reaves, Antonio Bianchi, William Enck, Alexandros Kapravelos, and Aravind Machiry. 2023. ARGUS: A Framework for Staged Static Taint Analysis of GitHub Workflows and Actions. In *32nd USENIX Security Symposium (USENIX Security 23)*. USENIX Association, Anaheim, CA, 6983–7000.
- [14] Owain Parry, Gregory M Kapfhammer, Michael Hilton, and Phil McMinn. 2021. A survey of flaky tests. *ACM Transactions on Software Engineering and Methodology (TOSEM)* 31, 1 (2021), 1–74.
- [15] Pooya Rostami Mazrae, Tom Mens, Mehdi Golzadeh, and Alexandre Decan. 2023. On the usage, co-usage and migration of CI/CD tools: A qualitative analysis. *Empirical Software Engineering* 28, 2 (2023), 52.
- [16] Xinyu Zhang, Siddharth Muralee, Sourag Cherupattamoolayil, and Aravind Machiry. 2024. On the Effectiveness of Large Language Models for GitHub Workflows. ACM, New York, NY, USA.